

ĐỒ ÁN LẬP TRÌNH MẠNG

Multi-Client Desktop Chat Application via TCP

Thiết Kế Kiến Trúc Hệ Thống & Hiện Thực Phần Mềm

Phân Chia Vai Trò Nhóm

Báo cáo chi tiết luồng xử lý và kiến trúc kỹ thuật phía Server và Client

Kiến Trúc TCP Hoạt Động

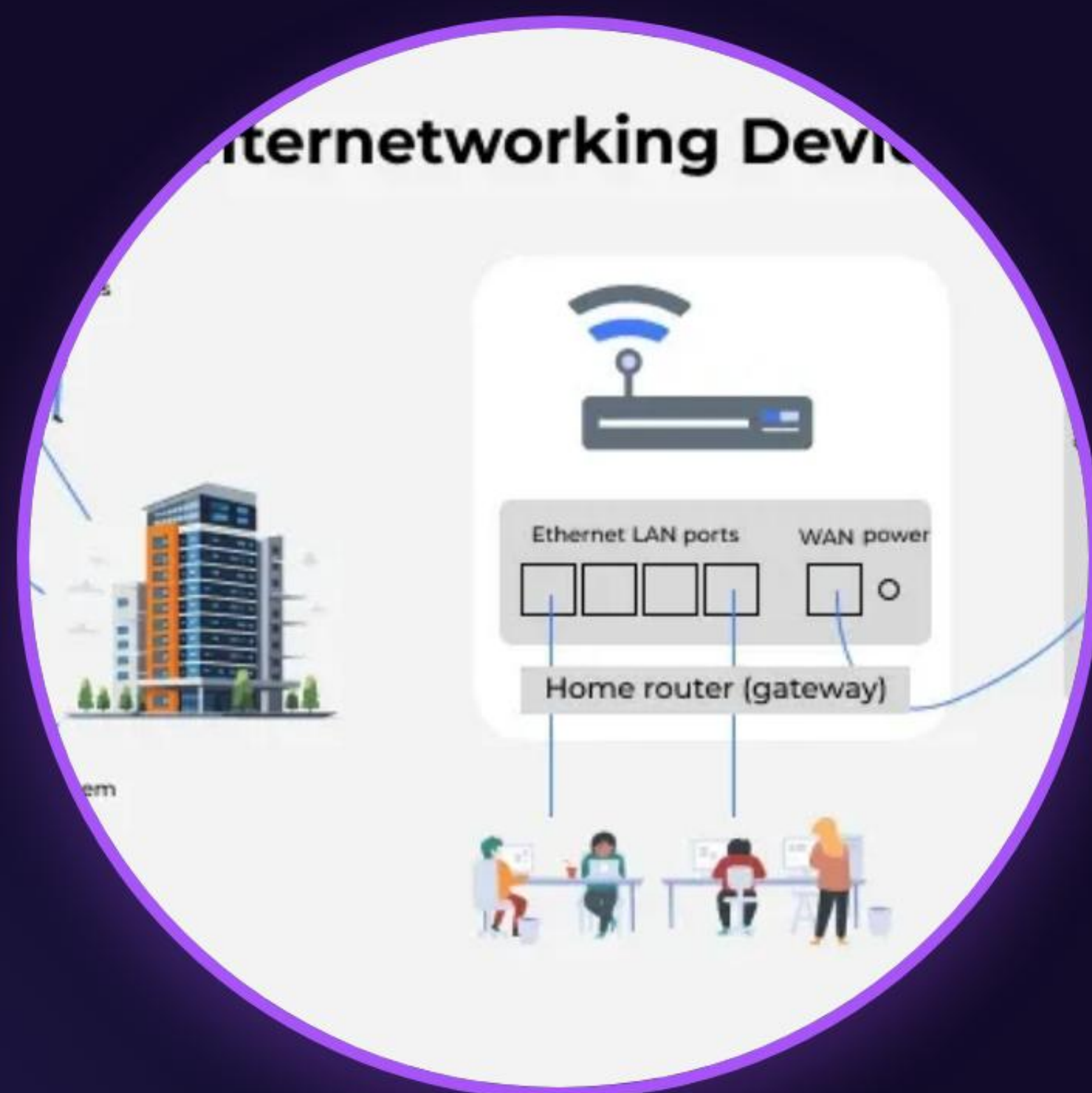
1. Giao Thức Kết Nối

Ứng dụng dựa trên giao thức TCP hướng kết nối tin cậy (Connection-oriented), đảm bảo truyền tải dữ liệu chat không bị thất thoát gói tin hoặc sai lệch thứ tự truyền.

2. Luồng Client-Server

Server lắng nghe các yêu cầu kết nối từ nhiều Client đồng thời. Mỗi kết nối Client được gán cho một tiểu trình (Thread) riêng biệt để xử lý gửi nhận song song.

Mô Hình Sockets Tổng Quan



Giao Tiếp Điểm - Điểm

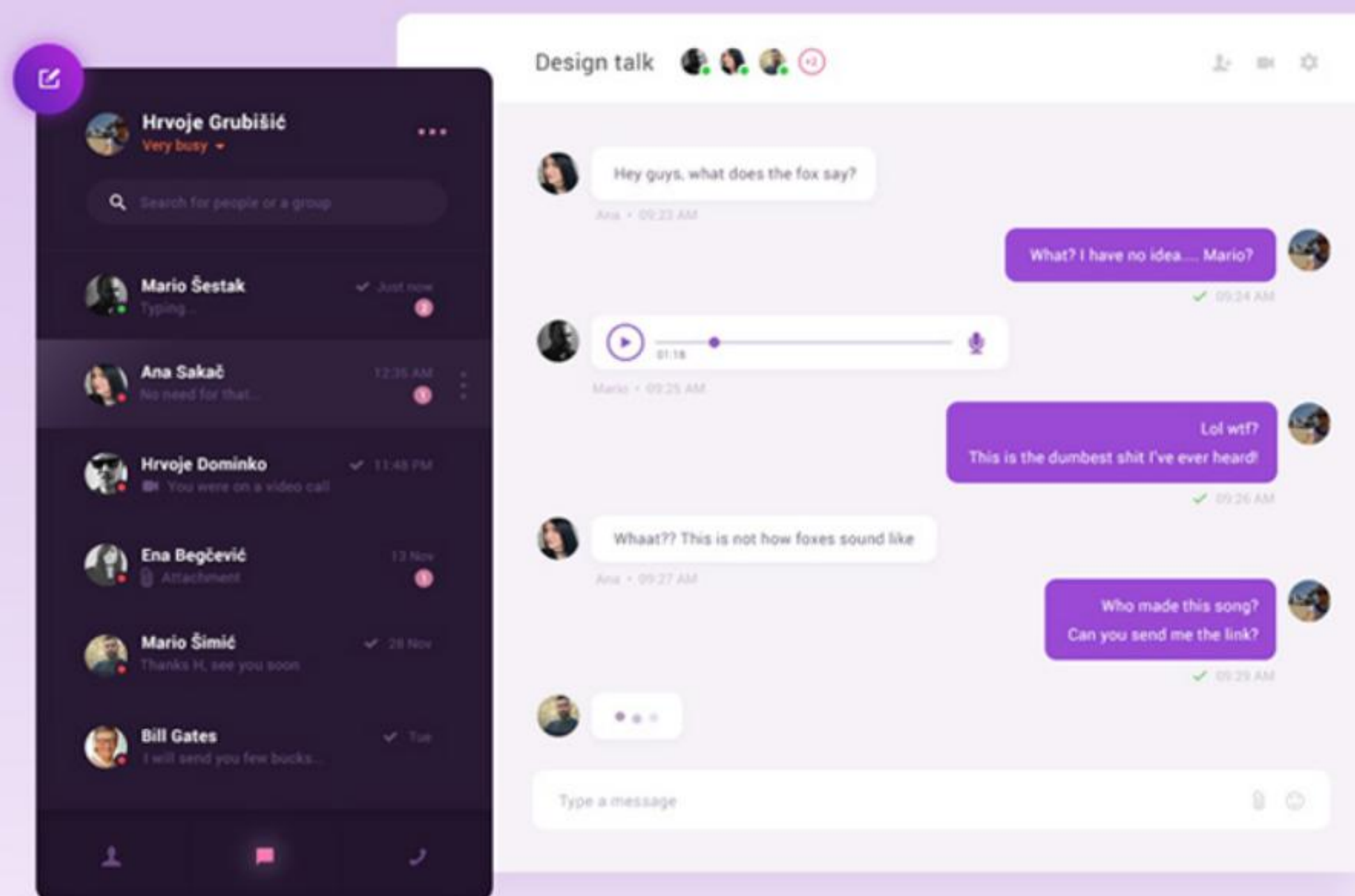
Mô hình Socket TCP định hình một đường truyền dữ liệu hai chiều (Full Duplex) liên tục. Khi một Client gửi thông điệp, Server nhận được, phân tích gói tin gói gọn trong lớp `NetworkPacket` và broadcast (truyền phát) tới các client khác đang kết nối.

Phần Xử Lý Phía Client

Kiến Trúc WPF, Giao Diện Người Dùng & Luồng Xử Lý Logic Network Client

Kiến Trúc WPF Client

- ❖ **Công nghệ thiết kế UI:** Sử dụng WPF (Windows Presentation Foundation) viết bằng XAML để dựng giao diện phong cách Desktop phẳng, hiện đại.
- ❖ **C# TcpClient:** Lớp thư viện chuẩn trong System.Net.Sockets được tích hợp để thiết lập kết nối và quản lý Network Stream một cách tối ưu.
- ❖ **Mô hình bất đồng bộ (Async/Await):** Giúp xử lý các tác vụ mạng tốn thời gian mà không gây hiện tượng treo hay giật lag giao diện chính (UI Thread).

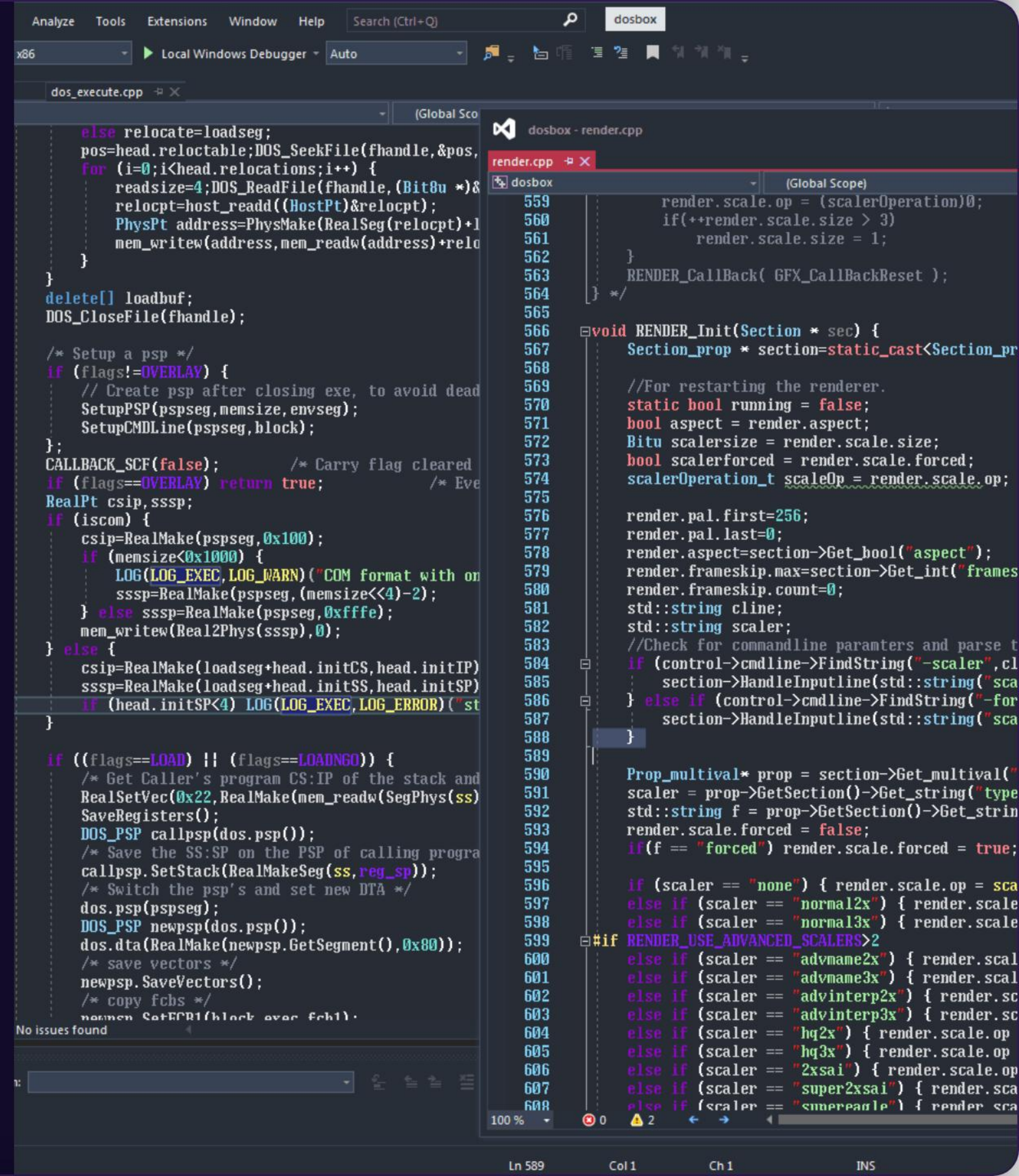


Logic Kết Nối Của Client

Xử Lý Ở ChatClientLogic.cs

Sử dụng TcpClient để thực hiện phương thức kết nối phi chặn đến Server IP và Port chỉ định.

Một khi kết nối thành công, Client khởi tạo luồng dữ liệu NetworkStream để sẵn sàng lắng nghe và ghi byte thông tin trao đổi.



Nhiệm Vụ Của Client



1. Kết Nối Mạng

Kết nối đến Server, bắt tay bắt đầu phiên chat và quản lý tự động ngắt kết nối an toàn.



2. Gửi Đóng Gói

Chuyển đổi văn bản nhập vào thành đối tượng `NetworkPacket`, serialize thành byte rồi ghi vào Stream.



3. Lắng Nghe & Cập Nhật

Sử dụng luồng phụ chạy ngầm để liên tục nhận dữ liệu từ Server, deserialize rồi đẩy an toàn lên UI.

Chỉ Số Hiệu Năng Phía Client

< 15ms

Độ Trễ Phản Hồi Trung Bình

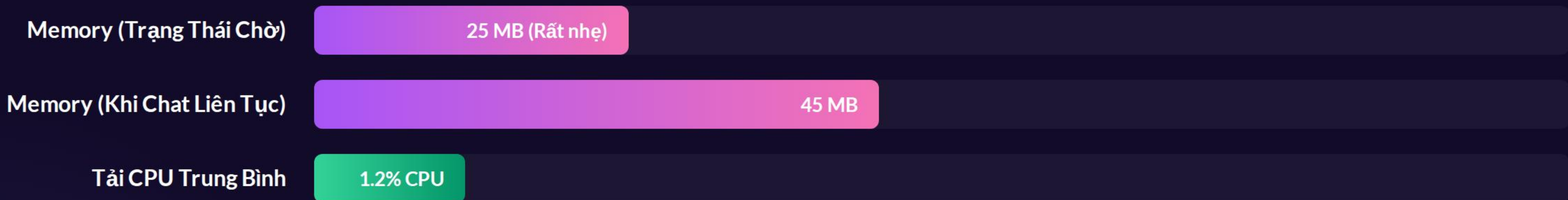
Tốc Độ Tối Ưu Tần Suất Cao

Nhờ sử dụng giao thức truyền nhận Socket thô không qua các lớp overhead của HTTP, gói tin đi trực tiếp từ card mạng Client đến Server giúp tối ưu hóa thời gian hiển thị tin nhắn gần như ngay lập tức (Real-time) trên máy khách.

So Sánh Phương Án Thiết Kế

Đặc Tính Kỹ Thuật	TCP Socket (Lựa Chọn Dự Án)	UDP Socket (Thay Thế)	HTTP Web API
Độ Tin Cậy Dữ Liệu	Đảm bảo tuyệt đối (Cơ chế ACK)	Có thể mất gói, sai thứ tự tin	Cao (Chạy trên nền TCP)
Thời Gian Trễ (Latency)	Cực kỳ thấp (Duy trì kết nối trực tiếp)	Thấp nhất (Không bắt tay)	Cao (Mở/Đóng connection liên tục)
Độ Khó Hiện Thực (Client)	Trung bình (Cần viết luồng đọc ngầm)	Trung bình	Dễ (Sử dụng HttpClient)

Mức Tiêu Thụ Tài Nguyên Phía Client



Ứng dụng WPF kết hợp cơ chế Thread-Pooling bất đồng bộ giúp giảm thiểu tối đa tài nguyên sử dụng trên RAM và CPU của máy khách Client.

Tiến Độ Thực Hiện Nhóm



Xin Cảm Ơn!

Hỏi & Đáp về Hệ Thống Chat Multi-Client TCP

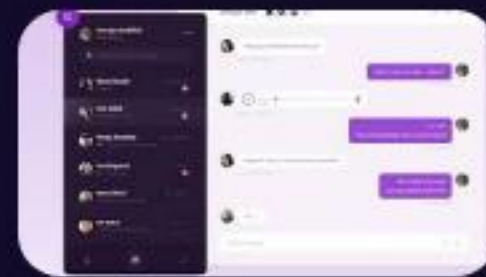
Trình bày bởi Thành viên Phụ trách Client | Môn Lập Trình Mạng

Image Sources



https://media.geeksforgeeks.org/wp-content/uploads/20250405120152143475/78_.webp

Source: www.geeksforgeeks.org



<https://cdn.dribbble.com/users/194964/screenshots/3144295/chat-module.jpg>

Source: muz.li



<https://i.redd.it/b7vl747an1vc1.png>

Source: www.reddit.com